1) For a given numpy array, how will you change the dimensions to 5 using the existing parameters in the numpy array? my_array = numpy.array([[[[1,3,4]]]])

Answer: numpy.array(my_array, ndmin=5)

Explanation: The `ndmin` parameter ensures the array has at least the specified number of dimensions. Setting `ndmin=5` forces the array to expand dimensions up to 5D, even if original array has fewer.

2) For the given numpy arrays, Array1 and Array2, what will be the dot product? Array1 = array([[1, 2, 3], [4, 5, 6]]) Array2 = array([[2, 3],[3, 2]])

Answer: ValueError

Explanation: Dot product requires inner dimensions to match (m×n · n×p). Here Array1 is (2×3) and Array2 is (2×2), which are incompatible. Hence, NumPy raises a ValueError.

3) In a given numpy array, using slicing operations print the reverse of the array. array = np.array([10,3,1,203,404,204,20,302,30,402,192])

Answer: array[::-1]

Explanation: The slicing step `-1` reverses the array order. `array[::-1]` returns a new array with elements from the last to the first.

4) What is the output of the below code? a = [[1, 0], [0, 1]] b = [[4, 1], [2, 2]] np.dot(a, b)

Answer: array([[4, 1],[2, 2]])

Explanation: Since `a` is the identity matrix, the dot product results in `b` itself. The multiplication preserves values of matrix `b`.

5) What is the output of the following code? a = np.arange(3*4*5*6).reshape((3,4,5,6)) b = np.arange(3*4*5*6)[::-1].reshape((5,4,6,3)) np.dot(a, b)[2,3,2,1,2,2]

Answer: ValueError

Explanation: The dot product requires matching last dimension of first array with second-to-last of second array. Here shapes are incompatible, so it throws ValueError.

6) What will be output for the following code? a = np.array([3, 7, 32], dtype=complex) print(a)

Answer: [3.+0.j 7.+0.j 32.+0.j]

Explanation: When dtype is set to `complex`, integers are converted to complex numbers with imaginary part `0.j`. Hence all numbers appear in complex form.

7) How to replace all values greater than 30 with 30 and less than 10 with 10? a = np.random.uniform(1,50,20)

Answer: Both np.clip(a, a_min=10, a_max=30) and np.where(...)

Explanation: `np.clip` restricts values to a range [10,30]. Alternatively, nested `np.where` can apply the same logic manually. Both methods work correctly.

8) For the given arrays, if we stack them row wise, what will be the output? array1 = np.array([[1,2],[3,4]]) array2 = np.array([[1,2],[3,4],[5,6],[7,8],[9,10]])

Answer: array([[1, 2],[3, 4],[1, 2],[3, 4],[5, 6],[7, 8],[9, 10]])

Explanation: Row-wise stacking concatenates along axis 0, so the rows of `array2` are appended after `array1`.

9) For a given numpy array, how will you insert a new value before 404? array = np.array([... ,404,...])

Answer: numpy.insert(array, np.where(array==404)[0][0], Elem_to_be_inserted)

Explanation: `np.where` finds the index of 404. `numpy.insert` inserts new elements at that index, shifting 404 and elements after it to the right.

10) Create a 3x3 matrix using numpy, and all the values must be constant k.

Answer: numpy.full((3,3), k)

Explanation: `np.full` creates an array of given shape filled with a constant value `k`. This is better than `ones` or `zeros` for custom constants.

11) For the given bubble sort implementation, what will be the output? my_array = np.array([20,14,25,16,45,60,12,9])

Answer: array([60, 45, 25, 20, 16, 14, 12, 9])

Explanation: The bubble sort swaps if the left element is smaller than the right. This results in descending order sorting, hence the largest elements come first.

12) For a numpy array of shape (2,5), reshape it to (5,2).

Answer: array.reshape(5,2)

Explanation: `reshape` changes the array's dimensions without altering data. The total elements (10) remain the same, just arranged into (5,2).

13) What will be the shape of the array after flattening? Sample = np.array([[1,2],[3,4],[5,6],[7,8]])

Answer: array([1,2,3,4,5,6,7,8])

Explanation: Flattening converts a multi-dimensional array into 1D. Here the (4,2) array becomes a (8,) vector with sequential elements.

14) From the array, extract output array([2,5,8])? Sample = np.array([[1,2,3],[4,5,6],[7,8,9],[10,11,12],[13,14,15]])

Answer: array[0:3, 1]

Explanation: The slice `0:3` selects first 3 rows, and column index `1` selects the second column values → [2,5,8].

15) Perform horizontal stacking on the arrays. Array1 = np.arange(1,20,2) Array2 = np.array([1,2,3,4,5,6,7,8,9,10])

Answer: array([[1,3,5,7,9,11,13,15,17,19], [1,2,3,4,5,6,7,8,9,10]])

Explanation: `hstack` stacks arrays column-wise if they have the same number of rows. This results in a 2×10 matrix with both arrays as rows.

16) Stack two arrays column-wise. array1 = np.array([[1,2],[3,4]]) array2 = np.array([[5,6],[7,8]])

Answer: array([[1,2,5,6],[3,4,7,8]])

Explanation: Column-wise stacking appends columns side by side. So each row of array2 is added after array1's corresponding row.

17) Given vectors A and B, find the cross product. A = np.array([4,12,29]) B = np.array([13,21,4])

Answer: array([-561, 360, -72])

Explanation: The cross product of two 3D vectors is another vector perpendicular to both. NumPy computes it via determinant expansion.

18) Find correlation coefficient of vectors A and B. A = np.array([1,3,5,...,25]) B = np.array([0,2,4,...,24])

Answer: array([[1.,1.],[1.,1.]])

Explanation: Since B is a perfect linear transformation of A, they have perfect positive correlation. NumPy's `corrcoef` returns a 2×2 matrix of correlations.

19) For arr1= np.array([[3,2,1],[6,4,5],[8,7,9]]), find difference between max per column and min per row.

Answer: [7 3 2]

Explanation: Max across columns → [8,7,9], Min across rows → [1,4,7]. Subtracting gives [7,3,2].

20) For a given numpy array, how will you change the dimensions to 5? my_array = np.array([[[[1,3,4]]]])

Answer: numpy.array(my_array, ndmin=5)

Explanation: The `ndmin` parameter ensures the array has at least 5 dimensions by adding new axes as required.